UNIT-4

Regular Grammar

Regular grammar represents regular languages. It can be either right-linear or left-linear grammar.

Defn:-
A grammar $G=(V,T,P,S)$ is said to be right-linear if all productions are of the form

$$A \to xB,$$
$$A \to x$$

where $A, B \in V$ and $x \in T^*$.

A grammar is said to be left linear if all productions are of the form

$$A \to Bx$$
$$A \to x.$$

A regular grammar is either right linear or left linear.

=

NOTE:-
In a regular grammar, at most one variable can appear on RHS and it has to be right most or left most symbol of right hand side of any production.

Ex:- 1) $S \to abS|a$ Right linear $(ab)^*a$

2) $S \to S_1ab$
$S_1 \to S_1ab|S_2$
$S_2 \to a$
Left linear. $aab(ab)^*$

3) $S \rightarrow A$
$A \rightarrow aB \mid \epsilon$
$B \rightarrow Ab$

It is not regular grammar: the grammar is neither left linear nor right linear. But it is a linear grammar.

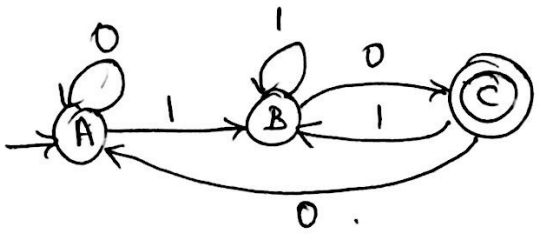
NOTE:-
Every regular grammar is linear, but not all linear grammars are regular.

Right Linear Grammar & DFA.

1. DFA to right linear grammar.

For each state transition of form $\delta(A, a) = B$, include production $A \rightarrow aB$.
If B is a final state, include $A \rightarrow a$.
Set of variables will be the states of DFA.
Start symbol will be start state.
Terminals will be Σ of DFA.

Ex:-
1)

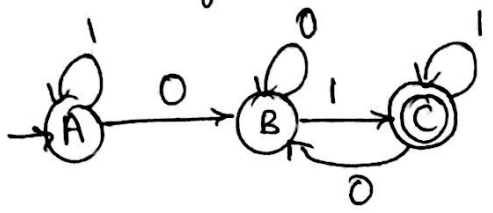


$A \rightarrow 0A \mid 1B$
$B \rightarrow 1B \mid 0C \mid 0$
$C \rightarrow 0A \mid 1B$

or

$A \rightarrow 0A \mid 1B$
$B \rightarrow 1B \mid 0C \mid 0$
$C \rightarrow 0A \mid 1B \mid \epsilon$

2) Obtain RLG for DFA.



Soln:-

Productions will be,

$A \rightarrow 1A$

$A \rightarrow 0B$

$B \rightarrow 0B$

$B \rightarrow 1C$

$B \rightarrow 1$

$C \rightarrow 1C$

$C \rightarrow 1$

$C \rightarrow 0B$

Transition

$\because \delta(A,1) = A$

$\because \delta(A,0) = B$

$\because \delta(B,0) = B$

$\delta(B,1) = C$

$\because C$ is a final state

$\delta(C,1) = C$

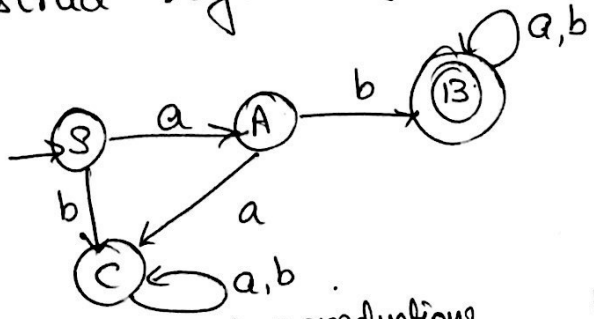
$\because C$ is final state

$\delta(C,0) = B$.

$\therefore G = (\{A,B,C\}, \{0,1\}, P, A)$.

where P has above productions is required RLG.

3. Construct regular grammar for FA.



Soln:-

Without ϵ productions

$S \rightarrow aA \mid bC$

$A \rightarrow bB \mid aC \mid b$

$B \rightarrow aB \mid bB \mid a \mid b$.

$C \rightarrow aC \mid bC$

or

With ϵ-productions

$S \rightarrow aA \mid bC$

$A \rightarrow bB \mid aC$

$B \rightarrow aB \mid bB \mid \epsilon$

$C \rightarrow aC \mid bC$.

to the D...

Right Linear grammar to DFA.

Given a RLG $G=(V,T,P,S)$. Construct DFA as below. Variables of G will be states of DFA. S will be start state. For each production of form,

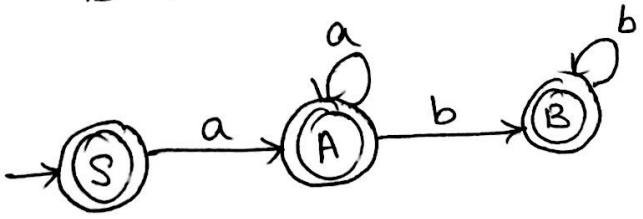
$A \to aB$, include transition $\delta(A,a)=B$.

If $A \to \epsilon$ is a production in G, then make A as final state.

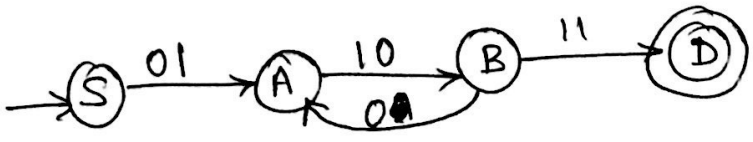
If $A \to a$ is a production in G, then check out which other production has $A \to aC$ form where $C \in V$. Then make C as a final state.
If not such production exists, add new final state D on transition a from A.

Ex:-

1) $S \to aA|\epsilon$
$A \to aA|bB|\epsilon$
$B \to bB|\epsilon$.



2) $S \to 01A$
$A \to 10B$
$B \to 0A|11$



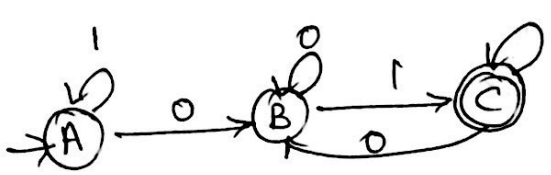
DFA to Left Linear Grammar.

1. Reverse DFA, make start state as final state & final state as start state & reverse directions of arrow.
2. obtain Right Linear grammar for reversed DFA.
3. Reverse productions of right linear grammar by reversing symbols on RHS to get left linear grammar.

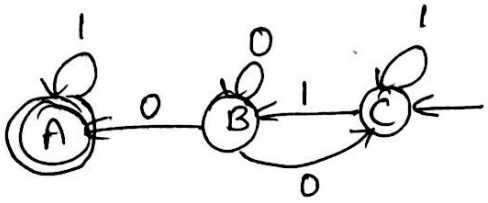
NOTE:- We assume that there is a single start state & single final state.

Ex:-

1)



Step1:- Reverse DFA.



Step2:- RLG for reversed DFA is

$C \rightarrow 1C|1B$
$B \rightarrow 0A|0C|0B$
$A \rightarrow 1A|\epsilon$

Step3:- Reverse productions on RHS.

$$P = \left\{ \begin{array}{l} C \rightarrow C1|B1 \\ B \rightarrow A0|C0|B0 \\ A \rightarrow A1|\epsilon . \end{array} \right\}$$

$\therefore G = (\{C, B, A\}, \{0, 1\}, P, C)$ is required LLG.

2) Convert to LLG.

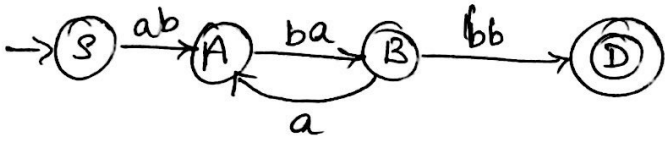
$S \rightarrow abA$

$A \rightarrow baB$

$B \rightarrow aA|bb$.

Soln:-

DFA for above RLG is



Step1:- Reverse DFA.

Step2:- Obtain RLG for reversed DFA.

$D \rightarrow bbB$

$B \rightarrow abA$

$A \rightarrow aB|baS$.

$S \rightarrow \epsilon$.

Step3:- Reverse RHS of each production.

$D \rightarrow Bbb$

$B \rightarrow Aba$

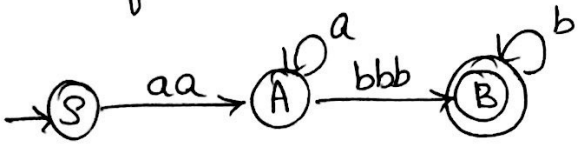
$A \rightarrow Ba|Sab$.

$S \rightarrow \epsilon$.

∴ Required LLG $G = (\{D, B, A, S\}, \{a, b\}, P, D)$

3) Obtain RLG for $L = \{a^n b^m \mid n \geq 2, m \geq 3\}$ also obtain its LLG.

Soln:- DFA for L is



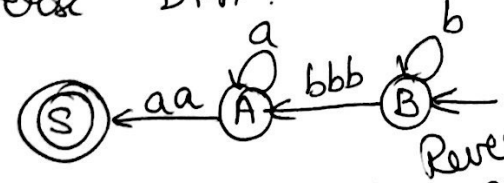
RLG is

$G = (\{S, A, B\}, \{a, b\}, P, S)$

$P = \{ S \to aaA$
$A \to aA \mid bbbB$
$B \to bB \mid \epsilon \}$

To obtain LLG.

Reverse DFA.



→ Reversing symbols on RHS of P' we get,

$P'' = \{ B \to Abbb \mid Bb$
$A \to Aa \mid Saa$
$S \to \epsilon \}$

$P' = \{ B \to bB \mid bbbA$
$A \to aA \mid aaS$
$S \to \epsilon \}$

$G' = (\{B, A, S\}, \{a, b\}, P'', B)$ is LLG

To derive aaabbbb using.

RLG

$S \Rightarrow aaA$ ∵ $S \to aaA$
$\Rightarrow aaaA$ ∵ $A \to aA$
$\Rightarrow aaabbbB$ ∵ $A \to bbbB$
$\Rightarrow aaabbbbB$ ∵ $B \to bB$
$\Rightarrow aaabbbb$ ∵ $B \to \epsilon$.

LLG

$B \Rightarrow Bb$ ∵ $B \to Bb$
$\Rightarrow Abbbb$ ∵ $B \to Abbb$
$\Rightarrow Aabbbb$ ∵ $A \to Aa$
$\Rightarrow Saabbbb$ ∵ $A \to Saa$
$\Rightarrow aaabbbb$ ∵ $S \to \epsilon$.

Conversion of LLG to RLG.

1. ~~Reverse RHS of productions & draw~~ DFA
2. Reverse the DFA, make start state as final state & final state as start state & reverse directions of arrow.
3. Write the productions

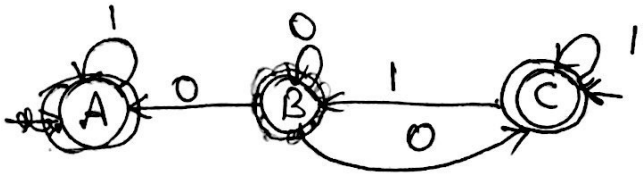
Ex:- Obtain LLG for given RLG.

1) $S \to C1 \mid B1$
$B \to A0 \mid C0 \mid B0$
$A \to A1 \mid \epsilon$

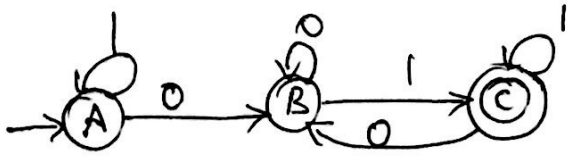
Soln:-

$C \to 1C \mid 1B \mid 1$
$B \to 0A \mid 0C \mid 0B$
$A \to 1A \mid \epsilon$.

DFA:



Reversed DFA:



RLG:

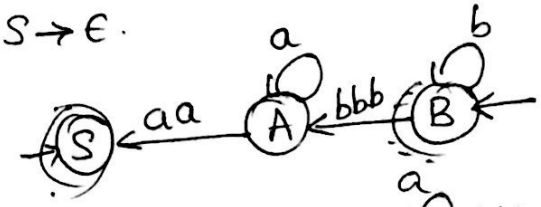
$A \to 1A \mid 0B$
$B \to 1C \mid 0B$
$C \to 0B \mid 1C \mid \epsilon$.

Obtain RLG equivalent of LLG

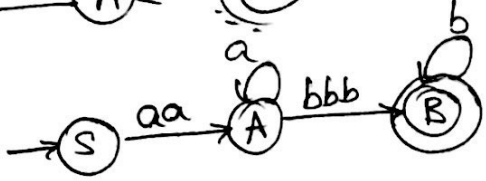
2) $B \to Bb \mid Abbb$
$A \to Aa \mid Saa$
$S \to \epsilon$.

Soln:- $B \to bB \mid bbbA$
$A \to aA \mid aaS$
$S \to \epsilon$.

DFA:



Reversed DFA:



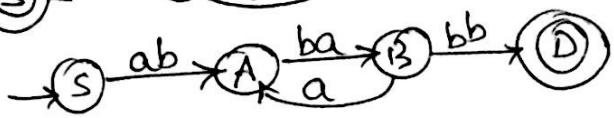
RLG: $S \to aaA$
$A \to aA \mid bbbB$
$B \to bB \mid \epsilon$.

3) $D \to Bbb$
$B \to Aba$
$A \to Ba \mid Sab$
$S \to \epsilon$.

Soln:- ~~DFA~~. $D \to bbB$
$B \to abA$
$A \to aB \mid baS$
$S \to \epsilon$.

DFA:

Reversed DFA:



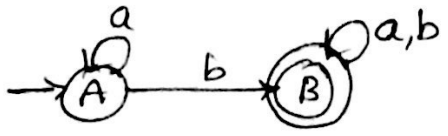
LLG: $S \to abA$
$A \to baB$
$B \to bbD \mid aA$
$D \to \epsilon$.

or

$S \to abA$
$A \to baB$
$B \to bb \mid aA$

1) Construct a regular grammar for

$a^*b\,(a+b)^*$

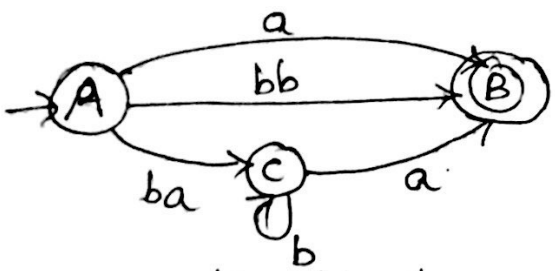


$A \to aA|bB$
$B \to aB|bB|\epsilon$.

or

$A \to aA|bB|b$
$B \to aB|a|bB|b$.

2) $a+bb+ba\,b^*a$



$A \to aB|bbB|baC$
$C \to bC|aB$
$B \to \epsilon$.

or

$A \to a|bb|baB$
$B \to bB|a$

Context-Sensitive Grammar

A Grammar G is said to be context Sensitive grammar if all productions are of the form,

$\alpha \rightarrow \beta$ with $|\beta| \geq |\alpha|$, $\alpha, \beta \in \{V \cup T\}^+$.

i.e., $|LHS| \leq |RHS|$ .or $|\alpha| \leq |\beta|$.

Any CFG without null productions is a context Sensitive grammar. But every context sensitive lang is not context free.

G is said to be CS iff it can be generated by a gramar in which every production has the form, $\alpha A \beta \rightarrow \alpha X \beta$, $\alpha, X, \beta \in (V \cup T)^+$ & $X \neq \phi$. $A \in V$.

$S \rightarrow aSBc \mid abc$
& $CB \rightarrow BC$
$bB \rightarrow bb$

Ex:- Obtain CSG for.

$L = \{a^n b^n c^n \mid n \geq 1\}$.

Soln:-

$S \rightarrow ABCS_1 \mid ABC$
$S_1 \rightarrow ABCS_1 \mid ABC$
$BA \rightarrow AB$
$A \rightarrow a$
$bB \rightarrow bb$
$CA \rightarrow AC$
$BA \rightarrow aa$
$bC \rightarrow bc$
$CB \rightarrow BC$
$aB \rightarrow ab$
$cC \rightarrow cc$ ..

(or)

$S \rightarrow abc \mid aAbc$
$Ab \rightarrow bA$
$Ac \rightarrow Bbcc$
$bB \rightarrow Bb$
$aB \rightarrow aa \mid aaA$.

or

$S \rightarrow aSBC \mid aBC$
$CB \rightarrow BC$
$aB \rightarrow ab$
$bB \rightarrow bb$
$bC \rightarrow bc$.

i.e, here variables A & B are used as messengers. An A is created on left of of b, travels to right to the first C, where it creates another bc. It

then sends the messenger B back to the left in order to create corresponding a.

To derive $a^3b^3c^3$.

$S \Rightarrow aAbc$ ∵ $S \to aAbc$

$\Rightarrow abAc$ ∵ $Ab \to bA$

$\Rightarrow abBbcc$ ∵ $Ac \to Bbcc$

$\Rightarrow aBbbcc$ ∵ $bB \to Bb$

$\Rightarrow aaAbbcc$ ∵ $aB \to aaA$

$\Rightarrow aabAbcc$

$\Rightarrow aabbAcc$

$\Rightarrow aabbBbccc$

$\Rightarrow aabBbbccc$

$\Rightarrow aaBbbbccc$

$\Rightarrow aaabbbccc$

2) $S \to aTb \mid ab$, $aT \to aaTb \mid ac$ } $L = \{ab \cup a^{n+1} c b^{n+1} \mid n \in N\}$

CFG for above L is $S \to aTb \mid ab$, $T \to aTb \mid c$.

CSL

$L_1 = \{a^n b^n c^n \mid n \geq 1\}$

$L_2 = \{a^{n!} \mid n \geq 0\}$

$L_3 = \{a^n \mid n = m^2, m \geq 1\}$ i.e. n is perfect square

$L_4 = \{a^p \mid p \text{ is prime}\}$

$L_5 = \{a^n \mid n \text{ is not prime}\}$

$L_6 = \{ww \mid w \in \{a,b\}^+\}$

$L_7 = \{w^n \mid w \in \{a,b\}^+, n \geq 1\}$

$L_8 = \{www^R \mid w \in (a,b)^+\}$

To generate $\underbrace{aabb}_{w}\underbrace{aabb}_{w}$

$S \Rightarrow \underline{B}X$
$\Rightarrow aBY\underline{X}$
$\Rightarrow a\underline{B}aX$
$\Rightarrow$ [illegible] $aaBY\underline{a}X$
$\Rightarrow aaBa\underline{Y}X$
$\Rightarrow aa\underline{B}aaX$
$\Rightarrow aabB\underline{Z}aaX$
$\Rightarrow aabBa\underline{Z}aX$
$\Rightarrow aabBaa\underline{Z}X$
$\Rightarrow aab\underline{B}aabX$
$\Rightarrow aabbaab\underline{X}$
$\Rightarrow aabb\ aabb.$

To generate $\underbrace{ab}_{w}\underbrace{ab}_{w}$

$S \Rightarrow \underline{B}X$
$\Rightarrow aBY\underline{X}$
$\Rightarrow a\underline{B}aX$
$\Rightarrow abaX$
$\Rightarrow abab.$

To generate $\underbrace{ba}_{w}\underbrace{ba}_{w}$

$S \Rightarrow \underline{A}G$
$\Rightarrow bAZ\underline{G}$
$\Rightarrow b\underline{A}bG$
$\Rightarrow ba\underline{b}G$
$\Rightarrow baba.$

CSG

2) $L = \{a^n b^m c^n a^m , m,n \geq 1\}$

$S \rightarrow Aa$
$A \rightarrow aAc | B | b$
$B \rightarrow bBX | b$
$Xc \rightarrow cX$
$Xa \rightarrow aa$

3) $L = \{a^n b^m c^m d^m , m,n > 0\}$

$S \rightarrow XY$
$X \rightarrow aXC | aC$
$Y \rightarrow BYd | Bd$
$CB \rightarrow BC$
$aB \rightarrow ab$
$bB \rightarrow bb$
$Cd \rightarrow cd$
$Cc \rightarrow cc$.

4) $L = \{WW \mid W \in \{0,1\}^*\}$

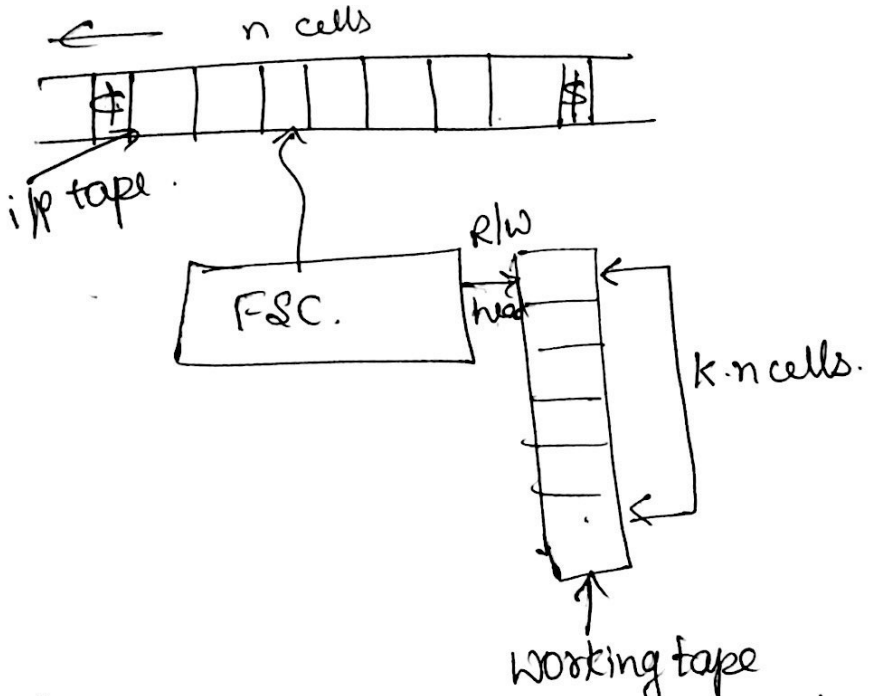
$S \rightarrow AC$ (→ end with a) $| BX$ (→ end with b)
$A \rightarrow aAY | bAZ | a$ create a on left
$B \rightarrow aBY | bBZ | b$ create b on left
$YC \rightarrow aC$
$YX \rightarrow aX$
$ZC \rightarrow bC$
$ZX \rightarrow bX$
$C \rightarrow a$
$X \rightarrow b$

$Ya \rightarrow aY$
$Yb \rightarrow bY$
$Za \rightarrow aZ$
$Zb \rightarrow bZ$
} move everything to right.

Linear Bounded Automata - accepts context sensitive language. LBA is a nondeterministic TM which has a single tape whose length is not infinite but bounded by a linear function of length of the i/p string.

$M = (Q, \Sigma, \Gamma, \delta, q_0, \#, ¢, \$, F)$

$¢, \$ \in \Sigma$.



The string $|w| = n-2$. then w can be recognised by an LBA if it can also be recognised by a TM using no more than kn cells of i/p tape, where k is a constant specified in the description of LBA. The value of k does not depend on i/p string but is purely a property of the m/c. There are 2 tapes one is i/p tape, other is working tape. Whenever we process any string in LBA, we assume that i/p string is enclosed within end markers ¢ & $. On i/p tape the tape head never prints & never moves left. On

the working tape. the head can modify the contents in any way, without any restriction.

ID of LBA: is denoted by (q, w, k), $q \in Q$, $w \in \Gamma$ k is an integer b/n $1 \leq k \leq n$. k changes to k-1 if R/W head moves to left & to k+1 if head moves to the right.

Language accepted by LBA.

$$L(M) = \{ w \mid (w \in \Sigma - \{¢, \$\})^*, ¢w\$ \vdash^* (q, \alpha, i) \}$$

for some $q \in F$, $1 \leq i \leq n$.